

Assignment no – 4

❖ Task-1 : Statistical Analysis- Mean, Median & Mode

Using Sample Superstore Dataset

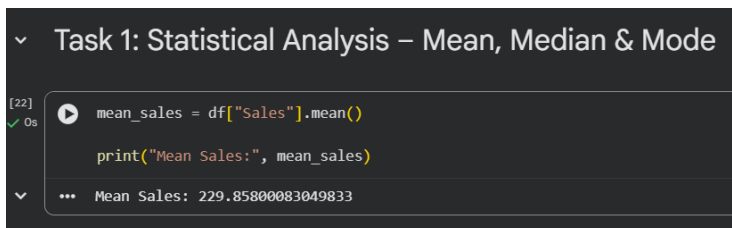
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
print("Libraries imported successfully!")
```

- **Mean**

Mean: Mean represents the average sales value of all orders in the dataset.

```
mean_sales = df["Sales"].mean()
print("Mean Sales:", mean_sales)
```

Output:



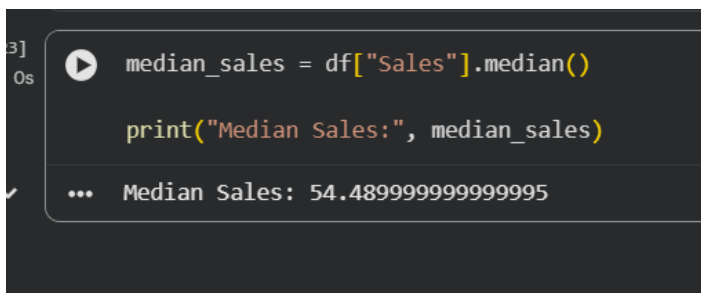
The screenshot shows a Jupyter Notebook interface. At the top, there is a section header 'Task 1: Statistical Analysis – Mean, Median & Mode'. Below it, a code cell is shown with the following code: `mean_sales = df["Sales"].mean()` and `print("Mean Sales:", mean_sales)`. The code is executed, and the output is displayed below the code cell: `... Mean Sales: 229.85800083049833`.

- **Median**

Median: Median represents the middle sales value after arranging all sales values in ascending order.

```
median_sales = df["Sales"].median()
print("Median Sales:", median_sales)
```

Output:



The screenshot shows a Jupyter Notebook interface. A code cell is shown with the following code: `median_sales = df["Sales"].median()` and `print("Median Sales:", median_sales)`. The code is executed, and the output is displayed below the code cell: `... Median Sales: 54.489999999999995`.

- **Mode**

Mode: Mode represents the most frequently occurring sales value in the dataset.

Output:

```
mode_sales = df["Sales"].mode()

print("Mode Sales:", mode_sales.tolist())

... Mode Sales: [12.96]
```

❖ Task -2 : Variance & Standard Deviation

```
variance = df["Sales"].var()

std_deviation = df["Sales"].std()

print("Variance:", variance)

print("Standard Deviation:", std_deviation)
```

Output:

```
Task 2: Variance & Standard

variance = df["Sales"].var()
std_deviation = df["Sales"].std()

print("Variance:", variance)
print("Standard Deviation:", std_deviation)

... Variance: 388434.45530807687
Standard Deviation: 623.2451005086818
```

➤ Explanation:

Variance measures how much the sales values differ from the mean. Standard deviation represents the spread of sales values around the mean. A higher standard deviation indicates greater variability in the data.

❖ Task -3 : Correlation

Sales VS Profit

```
correlation = df["Sales"].corr(df["Profit"])

print("Correlation between Sales and Profit:", correlation)
```

Output:

Task 3: Correlation & Probability

```
correlation = df["Sales"].corr(df["Profit"])

print("Correlation between Sales and Profit:", correlation)

... Correlation between Sales and Profit: 0.4790643497377062
```

Interpretation automatically

if correlation > 0:

```
print("The relationship is positive.")
```

elif correlation < 0:

```
print("The relationship is negative.")
```

else:

```
print("There is no linear relationship.")
```

Output:

```
if correlation > 0:
    print("The relationship is positive.")
elif correlation < 0:
    print("The relationship is negative.")
else:
    print("There is no linear relationship.")

... The relationship is positive.
```

➤ Explanation

Correlation measures the strength and direction of the relationship between two numerical variables. In this analysis, Sales and Profit are compared to determine whether higher sales are associated with higher profit.

❖ Task – 3 : Probability

Probability of an order belonging to Technology category.

```
technology_orders = (df["Category"] == "Technology").sum()
```

```
total_orders = len(df)
```

```
probability = technology_orders / total_orders
```

```
print("Technology Orders:", technology_orders)

print("Total Orders:", total_orders)

print("Probability:", probability)

print("Percentage:", probability * 100, "%")
```

Output:

```
▶ technology_orders = (df["Category"] == "Technology").sum()
total_orders = len(df)

probability = technology_orders / total_orders

print("Technology Orders:", technology_orders)
print("Total Orders:", total_orders)
print("Probability:", probability)
print("Percentage:", probability * 100, "%")

... Technology Orders: 1847
Total Orders: 9994
Probability: 0.18481088653191916
Percentage: 18.481088653191915 %
```

➤ Explanation

The probability represents the likelihood that a randomly selected order belongs to the Technology category.

❖ Task – 4 : Outlier Detection

```
Q1 = df["Sales"].quantile(0.25)
```

```
Q3 = df["Sales"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower_limit = Q1 - 1.5 * IQR
```

```
upper_limit = Q3 + 1.5 * IQR
```

```
print("Q1:", Q1)
```

```
print("Q3:", Q3)
```

```
print("IQR:", IQR)
```

```
print("Lower Limit:", lower_limit)
```

```
print("Upper Limit:", upper_limit)
```

Output:

```
Task 4: Outlier Detection

Q1 = df["Sales"].quantile(0.25)
Q3 = df["Sales"].quantile(0.75)

IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Limit:", lower_limit)
print("Upper Limit:", upper_limit)

... Q1: 17.28
    Q3: 209.94
    IQR: 192.66
    Lower Limit: -271.71000000000004
    Upper Limit: 498.93
```

Identify Outliers

```
outliers = df[
    (df["Sales"] < lower_limit) |
    (df["Sales"] > upper_limit)
]

print("Number of Outliers:", len(outliers))

outliers[["Sales", "Profit", "Category"]].head(20)

Number of Outliers: 1167
```

Output:

Number of Outliers: 1167

...		Sales	Profit	Category
1		731.9400	219.5820	Furniture
3		957.5775	-383.0310	Furniture
7		907.1520	90.7152	Technology
10		1706.1840	85.3092	Furniture
11		911.4240	68.3568	Technology
16		665.8800	13.3176	Office Supplies
24		1044.6300	240.2649	Furniture
27		3083.4300	-1665.0522	Furniture
35		1097.5440	123.4737	Technology
38		532.3992	-46.9764	Furniture
54		1029.9500	298.6855	Technology
67		1113.0240	111.3024	Office Supplies
72		831.9360	-114.3912	Furniture
117		787.5300	165.3813	Furniture
124		600.5580	-8.5794	Furniture
125		617.7000	-407.6820	Furniture
144		839.4300	218.2518	Office Supplies

Terminal

➤ **Explanation:**

The IQR method was used to identify potential outliers in the Sales column. Values below the lower limit or above the upper limit are considered potential outliers. Outliers can influence the mean, variance and other statistical analyses.

❖ **Task – 5: Matplotlib Visualization**

5.1 Line chart

```
plt.figure(figsize=(10,5))

plt.plot(df.index[:20], df["Sales"].iloc[:20], marker="o")

plt.title("Sales Trend for First 20 Orders")

plt.xlabel("Order Index")

plt.ylabel("Sales")

plt.show()
```

Output:



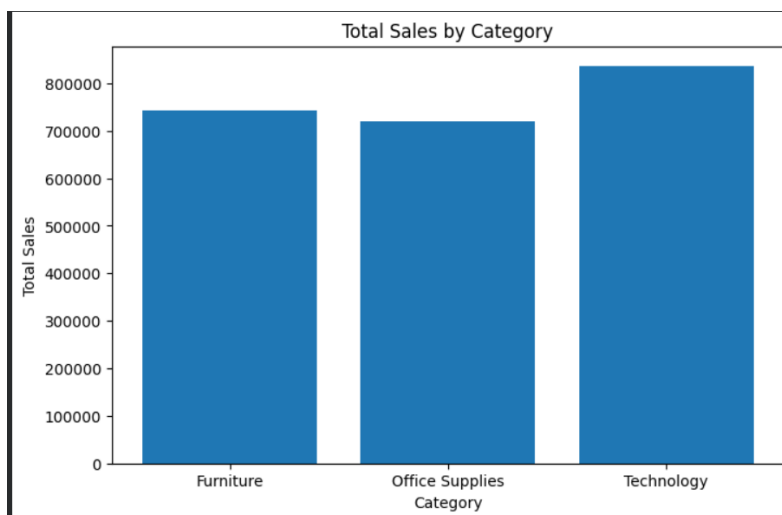
➤ **Explanation:**

The line chart shows the variation in sales across the first 20 orders.

5.2 Bar chart

```
category_sales = df.groupby("Category")["Sales"].sum()
plt.figure(figsize=(8,5))
plt.bar(category_sales.index, category_sales.values)
plt.title("Total Sales by Category")
plt.xlabel("Category")
plt.ylabel("Total Sales")
plt.show()
```

Output:



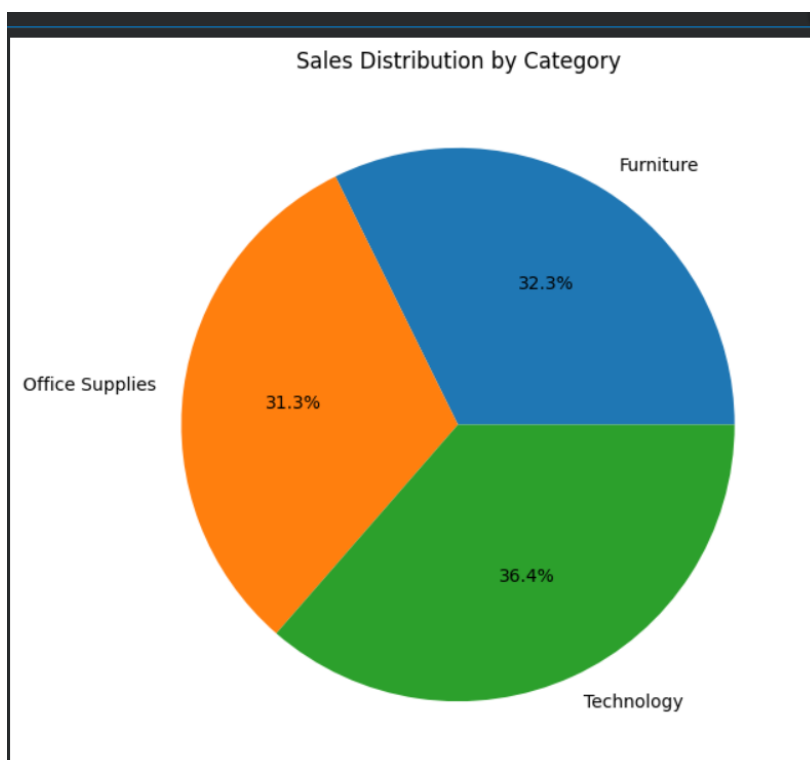
➤ **Explanation:**

The bar chart compares total sales across different product categories.

5.3 Pie chart

```
category_sales = df.groupby("Category")["Sales"].sum()
plt.figure(figsize=(7,7))
plt.pie(
    category_sales.values,
    labels=category_sales.index,
    autopct="%1.1f%%"
)
plt.title("Sales Distribution by Category")
plt.show()
```

Output:



➤ **Explanation:**

The pie chart shows the percentage contribution of each product category to total sales.

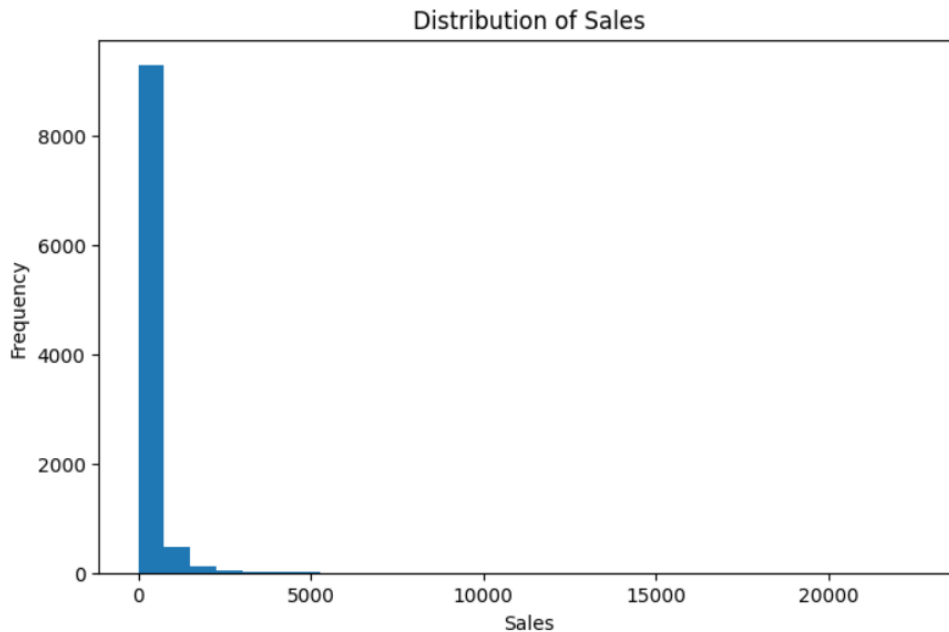
5.4 Histogram

```
plt.figure(figsize=(8,5))
plt.hist(df["Sales"], bins=30)
```



```
plt.title("Distribution of Sales")
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.show()
```

Output:



➤ **Explanation:**

The histogram displays the distribution and frequency of different sales values.

5.5 Scatter Plot

```
plt.figure(figsize=(8,5))
plt.scatter(df["Sales"], df["Profit"], alpha=0.5)
plt.title("Sales vs Profit")
plt.xlabel("Sales")
plt.ylabel("Profit")
plt.show()
```

Output:



➤ **Explanation:**

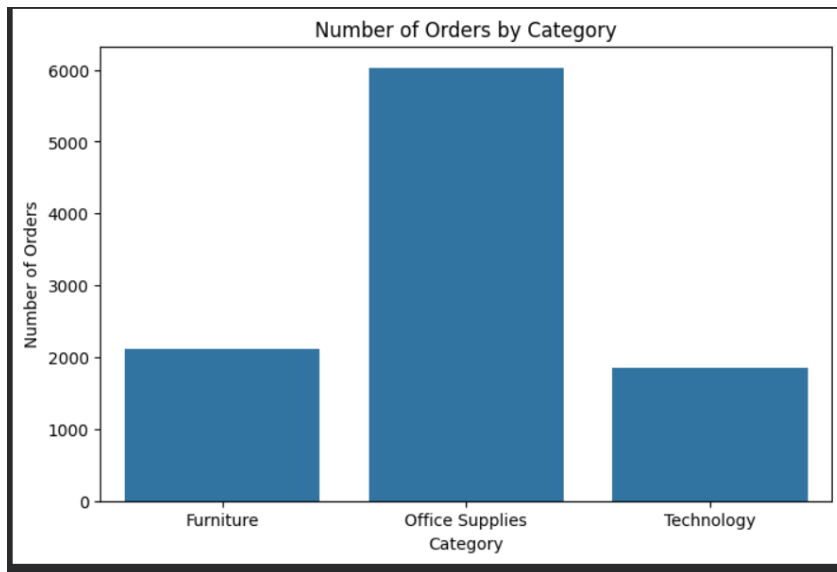
The scatter plot illustrates the relationship between Sales and Profit. It helps identify the direction of the relationship and possible unusual observations.

❖ **Task – 6 : Seaborn Visualization**

6.1 Count Plot

```
plt.figure(figsize=(8,5))  
sns.countplot(data=df, x="Category")  
plt.title("Number of Orders by Category")  
plt.xlabel("Category")  
plt.ylabel("Number of Orders")  
plt.show()
```

Output:



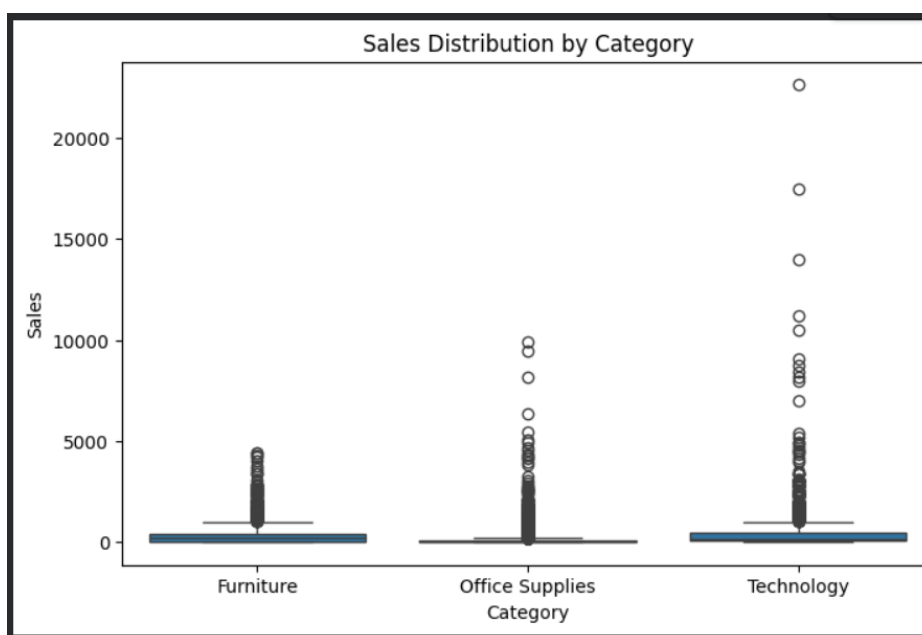
➤ **Explanation:**

The count plot shows the number of orders in each product category.

6.2 Box Plot

```
plt.figure(figsize=(8,5))
sns.boxplot(data=df, x="Category", y="Sales")
plt.title("Sales Distribution by Category")
plt.xlabel("Category")
plt.ylabel("Sales")
plt.show()
```

Output:



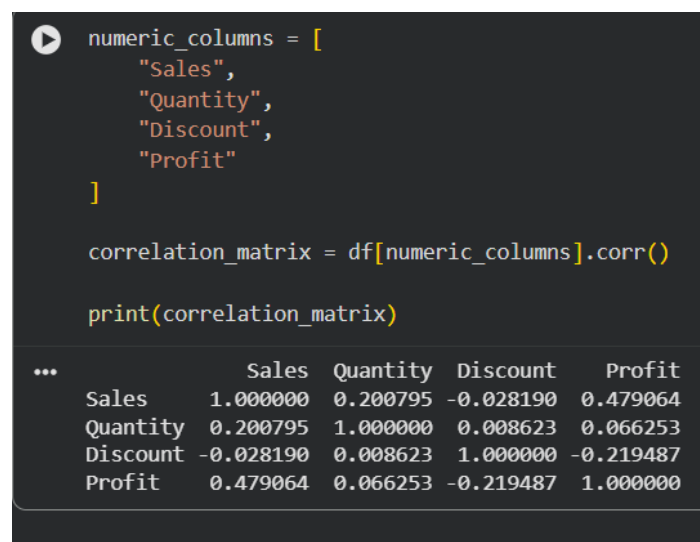
➤ **Explanation:**

The box plot shows the distribution, median, spread and potential outliers of sales across different categories.

6.3 Heatmap

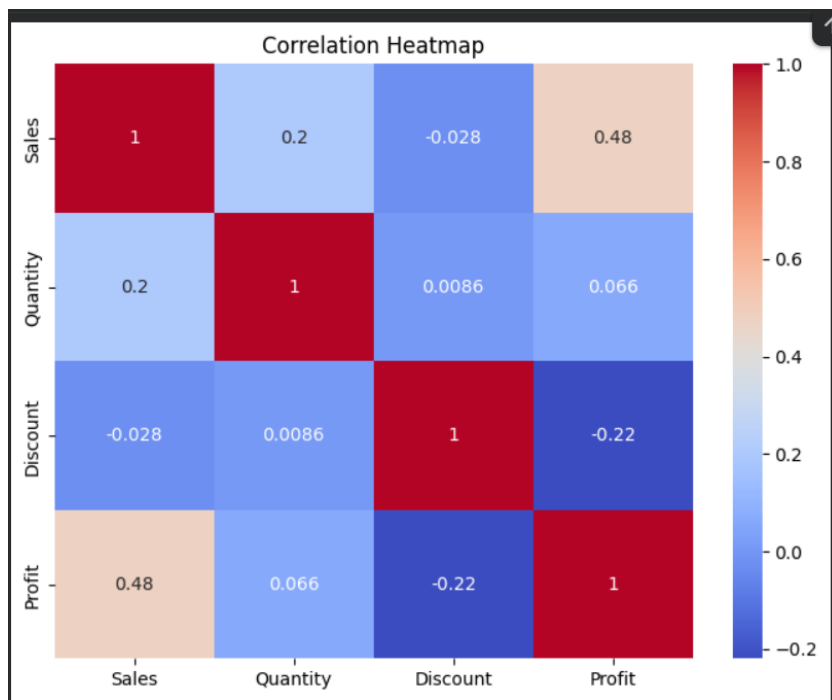
```
numeric_columns = [  
    "Sales",  
    "Quantity",  
    "Discount",  
    "Profit"  
]  
  
correlation_matrix = df[numeric_columns].corr()  
  
print(correlation_matrix)
```

Output:



```
plt.figure(figsize=(8,6))  
  
sns.heatmap(  
    correlation_matrix,  
    annot=True,  
    cmap="coolwarm"  
)  
  
plt.title("Correlation Heatmap")  
  
plt.show()
```

Output:



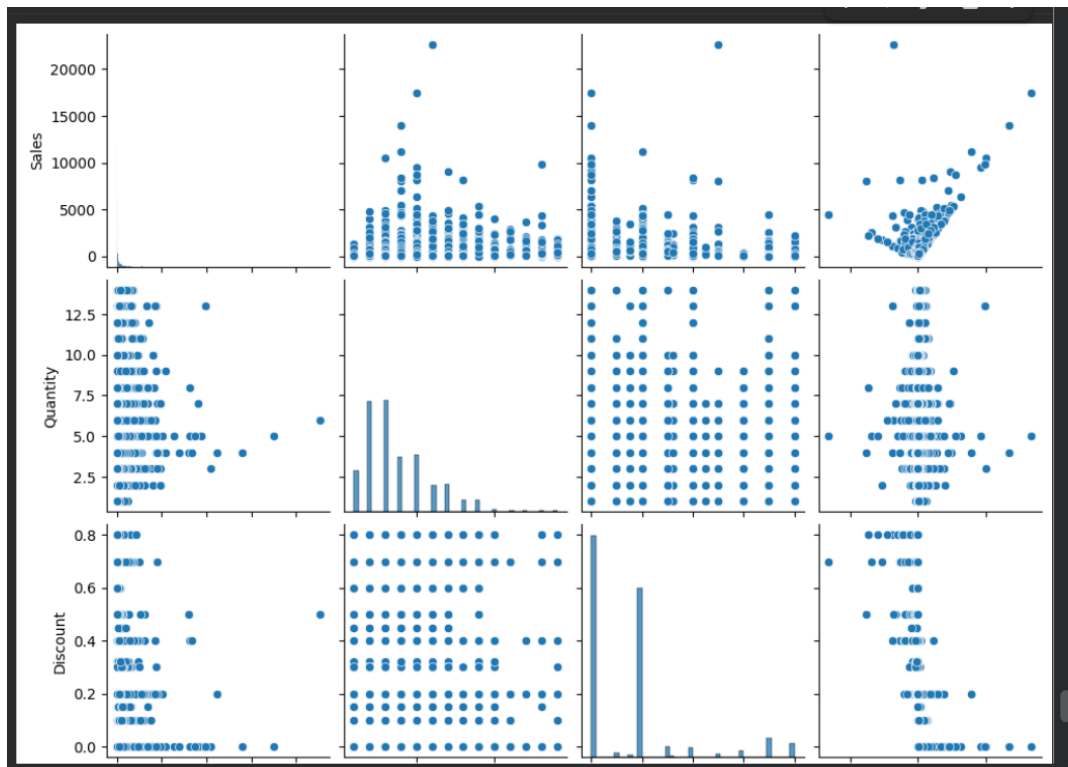
➤ **Explanation:**

The correlation heatmap displays the relationships between numerical variables. Values closer to +1 indicate strong positive relationships, while values closer to -1 indicate strong negative relationships.

6.4 Pair Plot

```
sns.pairplot(
    df[["Sales", "Quantity", "Discount", "Profit"]]
)
plt.show()
```

Output:



➤ **Explanation:**

The pair plot provides a visual representation of relationships between multiple numerical variables and helps identify patterns, correlations and possible outliers.

❖ **Task – 7 : EDA: Data Inspection and cleaning**

7.1 First 5 rows

`df.head()`

Output:

Task 7: Exploratory Data Analysis (EDA)

[41] 0s

df.head()

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region
0	1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	S
1	2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	...	42420	S
2	3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	...	90036	V
3	4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	S
4	5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	...	33311	S

Tables Terminal

7.2 Last 5 rows

df.head()

Output:

df.tail()

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Postal Code	Region
9989	9990	CA-2014-110422	2014-01-21	2014-01-23	Second Class	TB-21400	Tom Boeckenhauer	Consumer	United States	Miami	...	33131	S
9990	9991	CA-2017-121258	2017-02-26	2017-03-03	Standard Class	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	...	92626	S
9991	9992	CA-2017-121258	2017-02-26	2017-03-03	Standard Class	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	...	92626	S
9992	9993	CA-2017-121258	2017-02-26	2017-03-03	Standard Class	DB-13060	Dave Brooks	Consumer	United States	Costa Mesa	...	92626	S
9993	9994	CA-2017-119914	2017-05-04	2017-05-09	Second Class	CC-12220	Chris Cortes	Consumer	United States	Westminster	...	92683	S

5 rows x 21 columns

7.3 Dataset dimensions

```
print("Rows:", df.shape[0])  
print("Columns:", df.shape[1])
```

Output:

```
print("Rows:", df.shape[0])  
print("Columns:", df.shape[1])  
... Rows: 9994  
Columns: 21
```

7.4 Data types

```
Df.info()
```

Output:

```
df.info()  
... <class 'pandas.core.frame.DataFrame'>  
RangeIndex: 9994 entries, 0 to 9993  
Data columns (total 21 columns):  
#   Column                Non-Null Count  Dtype  
---  ---  
0   Row ID                9994 non-null  int64  
1   Order ID              9994 non-null  object  
2   Order Date            9994 non-null  datetime64[ns]  
3   Ship Date             9994 non-null  datetime64[ns]  
4   Ship Mode              9994 non-null  object  
5   Customer ID           9994 non-null  object  
6   Customer Name         9994 non-null  object  
7   Segment               9994 non-null  object  
8   Country               9994 non-null  object  
9   City                  9994 non-null  object  
10  State                 9994 non-null  object  
11  Postal Code           9994 non-null  int64  
12  Region                9994 non-null  object  
13  Product ID            9994 non-null  object  
14  Category              9994 non-null  object  
15  Sub-Category          9994 non-null  object  
16  Product Name          9994 non-null  object  
17  Sales                  9994 non-null  float64  
18  Quantity              9994 non-null  int64  
19  Discount              9994 non-null  float64  
20  Profit                9994 non-null  float64  
dtypes: datetime64[ns](2), float64(3), int64(3), object(13)  
memory usage: 1.6+ MB
```

7.5 Statistical summary

```
df.describe()
```

Output:

memory usage: 1.6+ MB

df.describe()

	Row ID	Order Date	Ship Date	Postal Code	Sales	Quantity	Discount
count	9994.000000	9994	9994	9994.000000	9994.000000	9994.000000	9994.000000
mean	4997.500000	2016-04-30 00:07:12.259355648	2016-05-03 23:06:58.571142912	55190.379428	229.858001	3.789574	0.156
min	1.000000	2014-01-03 00:00:00	2014-01-07 00:00:00	1040.000000	0.444000	1.000000	0.000
25%	2499.250000	2015-05-23 00:00:00	2015-05-27 00:00:00	23223.000000	17.280000	2.000000	0.000
50%	4997.500000	2016-06-26 00:00:00	2016-06-29 00:00:00	56430.500000	54.490000	3.000000	0.200
75%	7495.750000	2017-05-14 00:00:00	2017-05-18 00:00:00	90008.000000	209.940000	5.000000	0.200
max	9994.000000	2017-12-30 00:00:00	2018-01-05 00:00:00	99301.000000	22638.480000	14.000000	0.800
std	2885.163629	NaN	NaN	32063.693350	623.245101	2.225110	0.200

7.6 Missing Values

```
missing_values = df.isnull().sum()
```

```
print(missing_values)
```

Output:

```
missing_values = df.isnull().sum()

print(missing_values)
```

Row ID	0
Order ID	0
Order Date	0
Ship Date	0
Ship Mode	0
Customer ID	0
Customer Name	0
Segment	0
Country	0
City	0
State	0
Postal Code	0
Region	0
Product ID	0
Category	0
Sub-Category	0
Product Name	0
Sales	0
Quantity	0
Discount	0
Profit	0
dtype:	int64

7.7 Duplicate rows

```
duplicates = df.duplicated().sum()
```

```
print("Duplicate Rows:", duplicates)
```

Output:

```
▶ duplicates = df.duplicated().sum()

print("Duplicate Rows:", duplicates)

... Duplicate Rows: 0
```

Data Cleaning

Remove duplicates

```
df = df.drop_duplicates()

print("Dataset shape after removing duplicates:", df.shape)
```

Output:

```
▶ df = df.drop_duplicates()

print("Dataset shape after removing duplicates:", df.shape)

... Dataset shape after removing duplicates: (9994, 21)
```

Check missing values again

```
print(df.isnull().sum())
```

Output:

```
]
Ds
▶ print(df.isnull().sum())

... Row ID      0
  Order ID      0
  Order Date    0
  Ship Date     0
  Ship Mode     0
  Customer ID   0
  Customer Name 0
  Segment      0
  Country       0
  City          0
  State         0
  Postal Code   0
  Region        0
  Product ID    0
  Category      0
  Sub-Category  0
  Product Name  0
  Sales         0
  Quantity      0
  Discount      0
  Profit        0
dtype: int64
```

Check data types

Df.dtypes

Output:

Row ID	int64
Order ID	object
Order Date	datetime64[ns]
Ship Date	datetime64[ns]
Ship Mode	object
Customer ID	object
Customer Name	object
Segment	object
Country	object
City	object
State	object
Postal Code	int64
Region	object
Product ID	object
Category	object
Sub-Category	object
Product Name	object
Sales	float64
Quantity	int64

❖ Task -8 : EDA : Correlation & Insights

Business Insights

```
print("Total Sales:", df["Sales"].sum())
print("Total Profit:", df["Profit"].sum())
print("Total Quantity Sold:", df["Quantity"].sum())
print("\nSales by Category:")
print(df.groupby("Category")["Sales"].sum())
print("\nProfit by Category:")
print(df.groupby("Category")["Profit"].sum())
print("\nSales by Region:")
print(df.groupby("Region")["Sales"].sum())
```

Output:

```
... Total Sales: 2297200.8603000003
Total Profit: 286397.0216999999
Total Quantity Sold: 37873
```

Sales by Category:

```
Category
Furniture      741999.7953
Office Supplies 719047.0320
Technology      836154.0330
Name: Sales, dtype: float64
```

Profit by Category:

```
Category
Furniture      18451.2728
Office Supplies 122490.8008
Technology      145454.9481
Name: Profit, dtype: float64
```

Sales by Region:

```
Region
Central    501239.8908
East       678781.2400
South      391721.9050
West       725457.8245
Name: Sales, dtype: float64
```

8.1 Correlation Matrix

```
import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

numeric_columns = ["Sales", "Quantity", "Discount", "Profit"]

correlation_matrix = df[numeric_columns].corr()

print("Correlation Matrix:")

print(correlation_matrix)
```

Output:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

numeric_columns = ["Sales", "Quantity", "Discount", "Profit"]

correlation_matrix = df[numeric_columns].corr()

print("Correlation Matrix:")
print(correlation_matrix)

... Correlation Matrix:
      Sales  Quantity  Discount  Profit
Sales    1.000000  0.200795  -0.028190  0.479064
Quantity  0.200795  1.000000   0.008623  0.066253
Discount -0.028190  0.008623  1.000000  -0.219487
Profit    0.479064  0.066253  -0.219487  1.000000
```

8.2 Correlation Heatmap

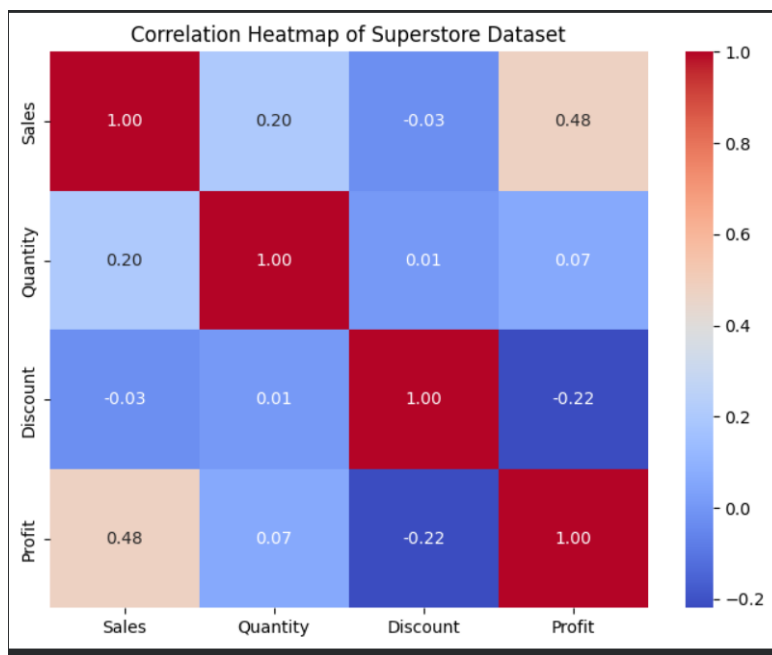
```
plt.figure(figsize=(8,6))
```

```
sns.heatmap(  
    correlation_matrix,  
    annot=True,  
    cmap="coolwarm",  
    fmt=".2f"  
)
```

```
plt.title("Correlation Heatmap of Superstore Dataset")
```

```
plt.show()
```

Output:



8.3 Important Relationships

```
print("Sales vs Profit Correlation:",  
      df["Sales"].corr(df["Profit"]))
```

```
print("Sales vs Quantity Correlation:",  
      df["Sales"].corr(df["Quantity"]))
```

```
print("Discount vs Profit Correlation:",
```

```
df["Discount"].corr(df["Profit"])
```

Output:

```
print("Sales vs Profit Correlation:",  
      df["Sales"].corr(df["Profit"]))  
  
print("Sales vs Quantity Correlation:",  
      df["Sales"].corr(df["Quantity"]))  
  
print("Discount vs Profit Correlation:",  
      df["Discount"].corr(df["Profit"]))  
  
... Sales vs Profit Correlation: 0.4790643497377062  
Sales vs Quantity Correlation: 0.20079477137389765  
Discount vs Profit Correlation: -0.21948745637176803
```

8.4 Patterns & Trends

Category sales:

```
category_sales = df.groupby("Category")["Sales"].sum()
```

```
print("Sales by Category:")
```

```
print(category_sales)
```

Output:

```
category_sales = df.groupby("Category")["Sales"].sum()  
  
print("Sales by Category:")  
print(category_sales)  
  
... Sales by Category:  
Category  
Furniture      741999.7953  
Office Supplies 719047.0320  
Technology      836154.0330  
Name: Sales, dtype: float64
```

Category Profit:

```
category_profit = df.groupby("Category")["Profit"].sum()
```

```
print("Profit by Category:")
```

```
print(category_profit)
```

Output:

```
category_profit = df.groupby("Category")["Profit"].sum()

print("Profit by Category:")
print(category_profit)
```

... Profit by Category:

Category	Profit
Furniture	18451.2728
Office Supplies	122490.8008
Technology	145454.9481

Name: Profit, dtype: float64

+ Code

Region – wise performance

```
region_sales = df.groupby("Region")["Sales"].sum()

print("Sales by Region:")

print(region_sales)
```

Output:

```
region_sales = df.groupby("Region")["Sales"].sum()

print("Sales by Region:")
print(region_sales)
```

... Sales by Region:

Region	Sales
Central	501239.8908
East	678781.2400
South	391721.9050
West	725457.8245

Name: Sales, dtype: float64

+ Co

```
region_profit = df.groupby("Region")["Profit"].sum()

print("Profit by Region:")

print(region_profit)
```

Output:

```
region_profit = df.groupby("Region")["Profit"].sum()

print("Profit by Region:")
print(region_profit)
```

... Profit by Region:

Region	Profit
Central	39706.3625
East	91522.7800
South	46749.4303
West	108418.4489

Name: Profit, dtype: float64

Correlation Analysis:

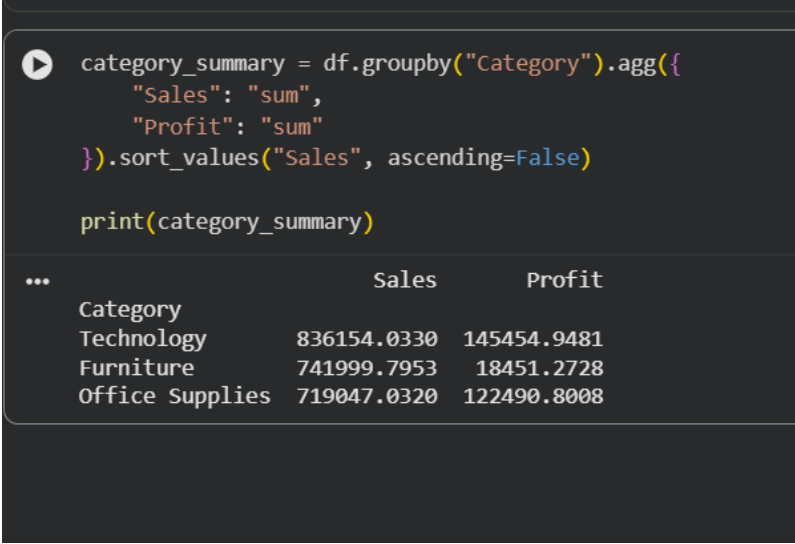
Correlation analysis was performed on Sales, Quantity, Discount and Profit to identify relationships between numerical variables. The correlation heatmap helps visualize the strength and direction of these relationships.

❖ Task -9 Business Recommendation

9.1 Category Performance.

```
category_summary = df.groupby("Category").agg({  
    "Sales": "sum",  
    "Profit": "sum"  
}).sort_values("Sales", ascending=False)  
  
print(category_summary)
```

Output:



The screenshot shows a Jupyter Notebook cell with a play button icon. The code defines a variable `category_summary` by grouping a dataframe `df` by 'Category' and aggregating 'Sales' and 'Profit' using the 'sum' function. The result is then sorted by 'Sales' in descending order. The output is printed, showing a table with three categories: Technology, Furniture, and Office Supplies, along with their respective total sales and profits.

	Sales	Profit
Category		
Technology	836154.0330	145454.9481
Furniture	741999.7953	18451.2728
Office Supplies	719047.0320	122490.8008

9.2 Region Performance

```
region_summary = df.groupby("Region").agg({  
    "Sales": "sum",  
    "Profit": "sum"  
}).sort_values("Profit", ascending=False)  
  
print(region_summary)
```

Output:

```
region_summary = df.groupby("Region").agg({
    "Sales": "sum",
    "Profit": "sum"
}).sort_values("Profit", ascending=False)

print(region_summary)
```

Region	Sales	Profit
West	725457.8245	108418.4489
East	678781.2400	91522.7800
South	391721.9050	46749.4303
Central	501239.8908	39706.3625

9.3 Sub-Category Performance

```
subcategory_summary = df.groupby("Sub-Category").agg({
    "Sales": "sum",
    "Profit": "sum"
}).sort_values("Profit", ascending=False)
```

```
print(subcategory_summary.head(10))
```

Output:

```
subcategory_summary = df.groupby("Sub-Category").agg({
    "Sales": "sum",
    "Profit": "sum"
}).sort_values("Profit", ascending=False)

print(subcategory_summary.head(10))
```

Sub-Category	Sales	Profit
Copiers	149528.030	55617.8249
Phones	330007.054	44515.7306
Accessories	167380.318	41936.6357
Paper	78479.206	34053.5693
Binders	203412.733	30221.7633
Chairs	328449.103	26590.1663
Storage	223843.608	21278.8264
Appliances	107532.161	18138.0054
Furnishings	91705.164	13059.1436
Envelopes	16476.402	6964.1767

Recommendation 1:

The company should focus more on high-performing categories because they generate higher sales and profit. Resources such as marketing and inventory can be prioritized for these **categories**.

Recommendation 2:

The company should review products or regions with high sales but low/negative profit and optimize pricing, discount strategies and operating costs.

Conclusion:

The Sample Superstore dataset was analyzed using statistical methods, data visualization and exploratory data analysis. Mean, median, mode, variance and standard deviation were calculated to understand the distribution of sales. Correlation analysis was performed to identify relationships between numerical variables. The IQR method was used to detect potential outliers. Matplotlib and Seaborn visualizations helped identify sales patterns, category-wise performance and relationships between variables. Overall, the analysis provides useful insights that can support business decision-making.

Assignment Completed Successfully.

Thank you.